

D&D 3.5e Prototype — Development Examples

Practical recipes for extending the game using the refactored service architecture.
All examples follow the patterns established during the service extraction refactor.

Table of Contents

1. [How to Add a New Feat](#)
2. [How to Add a New Spell](#)
3. [How to Add Status Effects](#)
4. [How to Add a New Service](#)
5. [Best Practices Checklist](#)

1. How to Add a New Feat

Step 1: Define the Feat

Open `Assets/Scripts/Character/FeatDefinitions.cs` and add the feat definition in `Init()`:

```
// In FeatDefinitions.Init()
RegisterFeat(new FeatData
{
    Name = "Weapon Expertise",
    Description = "+2 insight bonus to attack rolls with selected weapon (replaces
Weapon Focus for that weapon).",
    Prerequisites = new List<string> { "Base Attack Bonus +1" },
    Category = FeatCategory.Combat,
    IsStackable = false
});
```

Step 2: Add Logic to AttackCalculator

Open `Assets/Scripts/Combat/AttackCalculator.cs` and create a new method:

```
/// <summary>
/// Calculate Weapon Expertise bonus (custom feat).
/// +2 insight bonus to attack rolls.
/// </summary>
public static int GetWeaponExpertiseBonus(CharacterStats stats)
{
    if (stats == null || !stats.HasFeat("Weapon Expertise"))
        return 0;

    return 2; // +2 insight bonus
}
```

Step 3: Integrate into CalculateAllFeatModifiers

In the `CalculateAllFeatModifiers` method, add the new bonus:

```
// After existing Weapon Focus calculation:
int weaponExpertiseBonus = GetWeaponExpertiseBonus(stats);
result.WeaponFocusBonus += weaponExpertiseBonus; // Stacks with focus
```

Step 4: Add a FeatManager Helper

In `Assets/Scripts/Character/FeatManager.cs` :

```
public static bool HasWeaponExpertise(CharacterStats stats)
{
    return stats != null && stats.HasFeat("Weapon Expertise");
}
```

Step 5: Write a Test

Create or update a test file in `Assets/Scripts/Tests/Services/` :

```
private static void TestWeaponExpertise()
{
    var stats = MakeStats("WE_Test", feats: new[] { "Weapon Expertise" });
    int bonus = AttackCalculator.GetWeaponExpertiseBonus(stats);
    Assert(bonus == 2, "Weapon Expertise = +2", $"got {bonus}");
}
```

2. How to Add a New Spell

Step 1: Create Spell Data

Add the spell to the appropriate database file. For a spell starting with “M”, edit

`Assets/Scripts/Magic/Spells/Databases/SpellDatabase_M.cs` :

```
// In SpellDatabase_M.RegisterSpells()
Register(new SpellData
{
    SpellId = "improved_magic_missile",
    Name = "Improved Magic Missile",
    School = SpellSchool.Evocation,
    Level = 3,
    CastingTime = "1 standard action",
    Range = SpellRangeCategory.Medium,
    TargetType = SpellTargetType.SingleTarget,
    SavingThrow = "None",
    SpellResistance = true,
    Description = "As Magic Missile, but fires 1d4+1 missiles, each dealing 1d4+1
force damage.",
    DamageDice = "1d4+1",
    DamageType = "Force",
    BuffDurationRounds = 0, // Instantaneous
    ClassAvailability = new Dictionary<string, int>
    {
        { "Wizard", 3 },
        { "Sorcerer", 3 }
    }
});
```

Step 2: Register in SpellDatabase

Open `Assets/Scripts/Magic/SpellDatabase.cs` and add the registration call:

```
// In SpellDatabase.Init()
SpellDatabase_M.RegisterSpells(); // Already registered if file exists
```

Step 3: Handle Spell Resolution

For spells with special logic, add handling in `GameManager.ApplySpellBuff()` or `GameManager.PerformSpellCast()`. Use the existing services:

```
// In GameManager's spell handling section:
if (spell.SpellId == "improved_magic_missile")
{
    // Roll number of missiles: 1d4+1
    int missileCount = DiceService.D4("Improved Magic Missile count") + 1;
    int totalDamage = 0;

    for (int i = 0; i < missileCount; i++)
    {
        int dmg = DiceService.D4($"Magic Missile #{i + 1}") + 1;
        totalDamage += dmg;
    }

    // Use SpellResolutionService for pre-checks
    var preCheck = SpellResolutionService.RunPreCastChecks(caster, target, spell);
    if (!preCheck.SpellProceeds)
    {
        CombatUI?.ShowCombatLog(preCheck.LogMessage);
        return;
    }

    target.TakeDamage(totalDamage, "Force");
    CombatUI?.ShowCombatLog(
        $"🚀 {caster.Stats.CharacterName} fires {missileCount} improved missiles at "
+
        $"{target.Stats.CharacterName} for {totalDamage} force damage!");
    return;
}
```

Step 4: Use Centralized Services

Always use services for spell mechanics:

```
// ✅ CORRECT: Use DiceService
int damage = DiceService.RollMultiple(3, 6, "Fireball damage");

// ❌ WRONG: Raw Random.Range
int damage = Random.Range(3, 19);

// ✅ CORRECT: Use SpellResolutionService for pre-checks
var result = SpellResolutionService.RunPreCastChecks(caster, target, spell);

// ✅ CORRECT: Use SavingThrowResolver for saves
var save = SavingThrowResolver.ResolveReflexSave(target.Stats, dc, spell.Name);
```

3. How to Add Status Effects

Step 1: Create the Effect Data Class

Create a new file in `Assets/Scripts/Magic/StatusEffects/` :

```
// Assets/Scripts/Magic/StatusEffects/HasteEffectData.cs
using UnityEngine;

/// <summary>
/// D&D 3.5e Haste (PHB p.239):
/// - +1 attack bonus
/// - +1 dodge bonus to AC and Reflex saves
/// - +30 ft enhancement bonus to all movement modes
/// - One extra attack at full BAB during full attack
/// Duration: 1 round/level
/// </summary>
public class HasteEffectData
{
    public int AttackBonus = 1;
    public int ACBonus = 1;           // Dodge bonus
    public int ReflexBonus = 1;
    public int SpeedBonus = 30;       // In feet
    public bool GrantsExtraAttack = true;
    public int RemainingRounds;
    public int CasterLevel;

    public HasteEffectData(int casterLevel)
    {
        CasterLevel = casterLevel;
        // Duration: 1 round/level
        RemainingRounds = Mathf.Max(1, casterLevel);
    }
}
```

Step 2: Add Spell Data

In the appropriate `SpellDatabase_H.cs` :

```
Register(new SpellData
{
    SpellId = "haste",
    Name = "Haste",
    School = SpellSchool.Transmutation,
    Level = 3,
    TargetType = SpellTargetType.MultiTarget,
    Range = SpellRangeCategory.Close,
    SavingThrow = "Fortitude negates (harmless)",
    SpellResistance = true,
    DurationFormula = "1 round/level",
    Description = "One creature/level gains +1 attack, +1 AC, +30 speed, extra attack on full attack.",
    IsBuff = true,
    ClassAvailability = new Dictionary<string, int>
    {
        { "Wizard", 3 },
        { "Sorcerer", 3 },
        { "Bard", 3 }
    }
});
```

Step 3: Apply the Effect

Use `SpellApplicationService` helpers and `CombatLogger` for messages:

```
// In GameManager's ApplySpellBuff or spell handling:
if (spell.SpellId == "haste")
{
    int casterLevel = SpellApplicationService.GetEffectiveCasterLevel(caster);
    int duration = SpellApplicationService.CalculateDurationRounds(spell, caster-
Level);

    // Apply via StatusEffectManager
    var effect = _spellApplicationService.AddSpellEffect(target, spell, caster, caster
Level);

    // Apply stat modifications
    target.Stats.TemporaryAttackBonus += 1;
    target.Stats.DodgeBonus += 1;
    target.Stats.HasHaste = true;

    // Log via CombatLogger
    CombatLogger.Show(CombatLogger.FormatBuffApplied(
        caster.Stats.CharacterName,
        target.Stats.CharacterName,
        spell.Name,
        duration));

    return effect;
}
```

Step 4: Add Visual Indicator

In `Assets/Scripts/UI/StatusEffectIndicator.cs`, add display logic:

```
// In the icon generation section:
if (_character.Stats.HasHaste)
{
    icons.Add(new IconData
    {
        ShortLabel = "HA",
        FullName = "Haste",
        Tooltip = "Haste\n+1 attack, +1 AC, +30 speed\nExtra attack on full attack",
        Color = new Color(0.2f, 0.9f, 0.3f, 0.9f) // Green
    });
}
```

4. How to Add a New Service

Follow the `EconomyService` pattern:

Step 1: Create the Service Class

```
// Assets/Scripts/Services/MyNewService.cs
using System;
using UnityEngine;

public class MyNewService : MonoBehaviour
{
    // === STATE ===
    private GameManager _gameManager;
    private Func<CombatUI> _combatUIProvider;
    private CombatUI CombatUI => _combatUIProvider?.Invoke();

    // === LIFECYCLE ===
    public void Initialize(GameManager gm, Func<CombatUI> uiProvider)
    {
        _gameManager = gm;
        _combatUIProvider = uiProvider;
        Debug.Log("[MyNewService] Initialized");
    }

    public void Cleanup()
    {
        Debug.Log("[MyNewService] Cleaned up");
    }

    // === PUBLIC API ===
    public void DoSomething(CharacterController character)
    {
        // Always use DiceService for randomness
        int roll = DiceService.D20("MyNewService check");

        // Always use CombatLogger for messages
        CombatLogger.Show($"Something happened (rolled {roll})");
    }
}
```

Step 2: Register in GameManager

In `GameManager`, add the field and initialization:

```
// Field declaration (near other services)
[SerializeField] private MyNewService _myNewService;
public MyNewService MyNew => _myNewService;

// In the service initialization block:
_myNewService ??= gameObject.GetComponent<MyNewService>() ?? gameObject.AddComponent<MyNewService>();
_myNewService.Initialize(this, () => CombatUI);
```

Step 3: Delegate from GameManager

Replace `GameManager` logic with service calls:

```
// Before (in GameManager):
// int roll = Random.Range(1, 21);
// CombatUI?.ShowCombatLog($"Rolled {roll}");

// After:
_myNewService.DoSomething(character);
```

5. Best Practices Checklist

✓ Always Use Services

Task	Service	✗ Don't
Roll dice	<code>DiceService.D20()</code> , <code>DiceService.D6()</code>	<code>Random.Range(1, 21)</code>
Log combat	<code>CombatLogger.Show()</code> , <code>CombatLogger.Format*()</code>	<code>CombatUI?.ShowCombatLog()</code> directly
Saving throws	<code>SavingThrowResolver.ResolveSave()</code>	Manual d20 + modifier
Attack modifiers	<code>AttackCalculator.CalculateAllFeatModifiers()</code>	Inline feat checks
Spell pre-checks	<code>SpellResolutionService.RunPreCastChecks()</code>	Manual Blink/SR checks
Gold transactions	<code>EconomyService.SpendGold()</code> / <code>AddGold()</code>	Direct gold manipulation
Summon tracking	<code>SummoningService.RegisterSummonedCreature()</code>	Manual list management
Encounter XP	<code>EncounterService.CalculateEncounterXP()</code>	Manual CR-to-XP loops
Spell effects	<code>SpellApplicationService.AddSpellEffect()</code>	Direct <code>StatusEffectManager</code> access
Conditions	<code>SpellApplicationService.ApplyCondition()</code>	Direct <code>target.ApplyCondition()</code>

✓ Testing Requirements

- Every new feat needs a test in `Tests/Services/AttackCalculatorTests.cs`

- Every new service method needs a test in `Tests/Services/<ServiceName>Tests.cs`
- Tests follow the `Assert(condition, name, detail)` pattern
- Tests are static methods callable via `TestClass.RunAll()`
- Always call `TestHelpers.EnsureCoreDatabasesInitialized()` before tests

✓ File Organization

File Type	Location
Services	<code>Assets/Scripts/Services/</code>
Area effects	<code>Assets/Scripts/Magic/AreaEffects/</code>
Status effects	<code>Assets/Scripts/Magic/StatusEffects/</code>
Spell databases	<code>Assets/Scripts/Magic/Spells/Databases/</code>
Spell components	<code>Assets/Scripts/Magic/Components/</code>
Core spell files	<code>Assets/Scripts/Magic/</code> (root)
Combat mechanics	<code>Assets/Scripts/Combat/</code>
Tests	<code>Assets/Scripts/Tests/<Category>/</code>
AI profiles	<code>Assets/Scripts/AI/Profiles/</code>

✓ Code Style

```
// ✓ Always add XML doc comments to public methods
/// <summary>
/// Calculate bonus damage from Power Attack feat.
/// D&D 3.5e PHB p.98: Subtract from melee attack, add to melee damage.
/// </summary>
public static int CalculatePowerAttackDamage(CharacterStats stats, int value)

// ✓ Always add [context] prefix to Debug.Log
Debug.Log($"[ServiceName] Action description | key={value}");

// ✓ Always handle null inputs gracefully
if (character == null || character.Stats == null)
    return 0;

// ✓ Always use Mathf.Max/Mathf.Clamp for bounds
int duration = Mathf.Max(1, calculatedDuration);
int gold = Mathf.Max(0, newGoldValue);
```

✓ Service Extraction Pattern

When extracting logic from GameManager:

1. **Create** the service class as a `MonoBehaviour`
2. **Add** `Initialize()` with dependency injection (GameManager, CombatUI provider)
3. **Move** state (fields) and methods to the service
4. **Keep** delegation methods in GameManager for backward compatibility

5. **Wire** events for cross-service communication
 6. **Test** the service independently with unit tests
 7. **Document** the service in this file
-

Architecture Overview

```
GameManager (coordinator)
├── EconomyService      – Gold, shop, stash
├── SummoningService    – Summoned creature lifecycle
├── EncounterService     – Combat encounter lifecycle, XP
├── SpellApplicationService – Spell effect apply/remove/query
├── DiceService (static) – All random rolls
├── AttackCalculator (static) – Feat-based attack/damage mods
├── SpellResolutionService (static) – Blink/SR pre-checks
├── SavingThrowResolver (static) – Save calculations
├── CombatFlowService   – Attack execution pipeline
├── ConditionService     – Condition management
├── AIService           – NPC AI decision-making
├── TurnService          – Initiative and turn order
├── MovementService     – Movement and pathfinding
└── InputService        – User input handling
```